Operating Systems
Sheet 3 Solutions

Name: Yehia Said Gewily
ID: 21010705
Date: March 2026

1 What is a Process? (3.1)

1. Define: Process

A process is basically a program that is currently running. The difference is that a program is just passive code sitting on your disk, while a process is the live, active version of it loaded into memory. It includes the code itself, its data, and its execution context (things like the program counter and register values that tell the CPU exactly where it left off).

2 Process States (3.2)

2. Disadvantages of the two-state model and how to solve them

The Problem: The two-state model (just "Running" or "Not Running") is way too simple. It lumps together processes that are *ready* to run with processes that are *blocked* — waiting for an I/O device, for example. If the OS can't tell why a process isn't running, it wastes time looking at blocked processes that literally can't do anything yet.

The Fix: Add more specific states. A five-state or seven-state model lets the OS keep a separate "Ready" queue and a "Blocked" queue, so the scheduler only picks from processes that are actually ready to go.

3. Infinite `fork()` loop system call

a. Consequences of allowing it: This is essentially a fork bomb. The process doubles itself over and over, very quickly consuming all of the system's CPU time, memory, and

process table slots. The OS would freeze or crash.

b. Runtime controls to detect it: The OS can enforce per-user process quotas — a hard ceiling on how many processes one user can spawn. It could also watch for unusually rapid `fork()` calls and throttle a process if it goes over a threshold.

c. Impact on normal processes: These limits wouldn't really affect well-behaved programs. A normal application has no reason to spawn thousands of children in a fraction of a second, so only the runaway process would ever hit the cap.

d. Effect on OS design: The kernel would need extra bookkeeping to track the active process count per user, plus a small quota check every time `fork()` is called. It adds a tiny bit of overhead but it's worth it for stability.

4. Parent and child process destruction

a. Advantages and Disadvantages:

- **Auto-destroying children when parent exits:** Keeps the system clean — no orphan processes lingering around wasting resources. The downside is that you could accidentally kill a useful background task when the parent closes.
- **Letting children run independently:** Much more flexible. You can launch long-running services without worrying about their parent. The trade-off is that the OS has to "adopt" these orphan processes (typically re-parenting them to `init`) to ensure they're eventually cleaned up.

b. Real-world example: If you open a terminal (the parent) and use it to launch a background web server (the child), you want that server to keep running even after you close the terminal. Auto-destruction would kill it immediately.

5. Define: Suspended process and Zombie process

- **Suspended process:** A process that the OS has temporarily moved out of RAM and onto disk to free up memory. It simply can't run again until the OS decides to swap it back in.
- **Zombie process:** A process that has already finished and exited, but whose parent hasn't called `wait()` to collect its exit status. It's dead, but still holds a tiny slot in the process table until the parent acknowledges it.

6. Seven-state model dispatch policy dilemma

If we only ever dispatch from the in-memory "Ready" queue, we might ignore higher-priority processes sitting on disk. But if we always swap in from "Ready/Suspend" first, we risk constant disk thrashing.

A reasonable middle ground: Compare priorities. If the top suspended process has only a slightly higher priority than the top in-memory process, just leave it on disk and avoid the swapping delay. Only swap it in when the priority gap is large enough to justify the overhead — like if it's a critical system alert that really can't wait.

3 Process Description and Control (3.3 & 3.4)

7. Define: Process Control Block (PCB)

Think of the PCB as the process's ID card and status report combined. It's a data structure kept in the kernel that holds everything the OS needs to know about a process: its PID, current state, memory pointers, open file handles, and the exact CPU register snapshot from the last time it was paused. Without it, context switching would be impossible.

8. Process switching vs. Mode switching

- **Process Switching:** The OS completely pauses Process A and starts running Process B. This means saving Process A's entire context to its PCB, loading Process B's context, flushing caches, and updating memory maps. It's relatively expensive.
- **Mode Switching:** A process just needs the OS to do something on its behalf (like reading a file), so the CPU flips from user mode to kernel mode. It stays inside the same process. Only a privilege bit changes and a few registers are saved, so it's much faster than a full process switch.

9. Protecting the PCB from damage

PCBs are stored in protected kernel memory. Hardware-level memory protection (paging or segmentation) makes it physically impossible for a user-level program to read or overwrite that region. If a process wants to interact with its own PCB information, it has to go through a secure system call — there's no direct access.

10. Multiple blocked queues and priority handling

- **Why multiple queues?** Instead of dumping all blocked processes into one big pile, the OS maintains a separate queue per event type (one for disk, one for printer, etc.). When the printer finishes, the OS immediately knows exactly which processes to wake up without searching through the entire blocked list.
- **Handling priorities within a queue:** Each event queue keeps its waiting processes sorted by priority. When the event fires, the highest-priority process in that queue moves to Ready first, so urgent tasks aren't stuck behind low-priority ones.

4 UNIX SVR4 Process Management (3.6)

11. What happens when `fork()` is called?

Calling `fork()` makes the OS clone the current process. The child gets a new PID and a full copy of the parent's memory and PCB. After the clone, both the parent and child resume from the very next line of code. The key difference is the return value: the parent receives the child's PID, while the child receives `0`. That's how each one figures out which role it's playing.

5 General Questions

12. Priority-based preemptive scheduling (with aging)

Assumptions: Priority increases by 1 for every 5 ms a process spends waiting (aging). Context switches cost 2.5 ms and only happen when switching between different processes.

Execution Timeline:

- $\mathbf{t = 0}$: **P1** (priority 1) arrives and starts running.
- $\mathbf{t = 5}$: **P2** (priority 3) arrives. Since $3 > 1$, P2 preempts P1.
- $\mathbf{t = 5 \rightarrow 7.5}$: Context switch to P2.
- $\mathbf{t = 7.5}$: **P2** starts running (needs 7.5 ms).
- $\mathbf{t = 10}$: **P3** (priority 2) arrives. P2 is running at priority 3, so no preemption.

- **t = 15**: **P2 finishes.** Turnaround for P2 = 15 − 5 = **10 ms**.
  P4 (priority 2) also arrives. Aging check:
  - P1 waited ≈ 7.5 ms ⇒ priority = 1 + 1 = 2.
  - P3 waited 5 ms ⇒ priority = 2 + 1 = **3**.
  - P4 just arrived ⇒ priority = 2.

  P3 wins.
- **t = 15 → 17.5**: Context switch to P3.
- **t = 17.5**: **P3** starts running (needs 5 ms).
- **t = 22.5**: **P3 finishes.** Turnaround for P3 = 22.5 − 10 = **12.5 ms**.
  Aging check:
  - P1 waited 15 ms (from $t = 7.5$ to 22.5) ⇒ priority = 1 + 3 = **4**.
  - P4 waited 7.5 ms (from $t = 15$ to 22.5) ⇒ priority = 2 + 1 = 3.

  P1 wins.
- **t = 22.5 → 25**: Context switch to P1.
- **t = 25**: **P1** resumes with 10 ms remaining.
- **t = 35**: **P1 finishes.** Turnaround for P1 = 35 − 0 = **35 ms**.
- **t = 35 → 37.5**: Context switch to P4.
- **t = 37.5**: **P4** runs (needs 10 ms).
- **t = 47.5**: **P4 finishes.** Turnaround for P4 = 47.5 − 15 = **32.5 ms**.

13. Simplified process state transitions

a. Example events for each transition:

- **1 (READY → RUN):** The dispatcher picks this process and assigns it to the CPU.
- **2 (RUN → READY):** The process's time slice expires, or a higher-priority process arrives and preempts it.
- **3 (RUN → BLOCKED):** The running process requests a disk read and must wait for it to complete.
- **4 (BLOCKED → READY):** The disk finishes, so the process can run again and

moves back to the ready queue.

- **5 (READY → SUSPENDED):** The OS is low on RAM and swaps a ready-but-not-yet-running process out to disk.
- **6 (BLOCKED → SUSPENDED):** The OS swaps out a blocked process to free memory for more active ones.

b. Three impossible transitions:

- **READY → BLOCKED:** A process can't issue an I/O request unless it's actually executing on the CPU. A waiting process has no way to trigger a block event.
- **BLOCKED → RUN:** Once a blocked process's event completes, it goes to READY first. The dispatcher only picks from the ready queue — never directly from blocked.
- **RUN → SUSPENDED:** In standard models, a running process must first be preempted back to READY (or move to BLOCKED) before the OS can swap it out. You don't suspend something mid-execution.